

1 Explain the following.

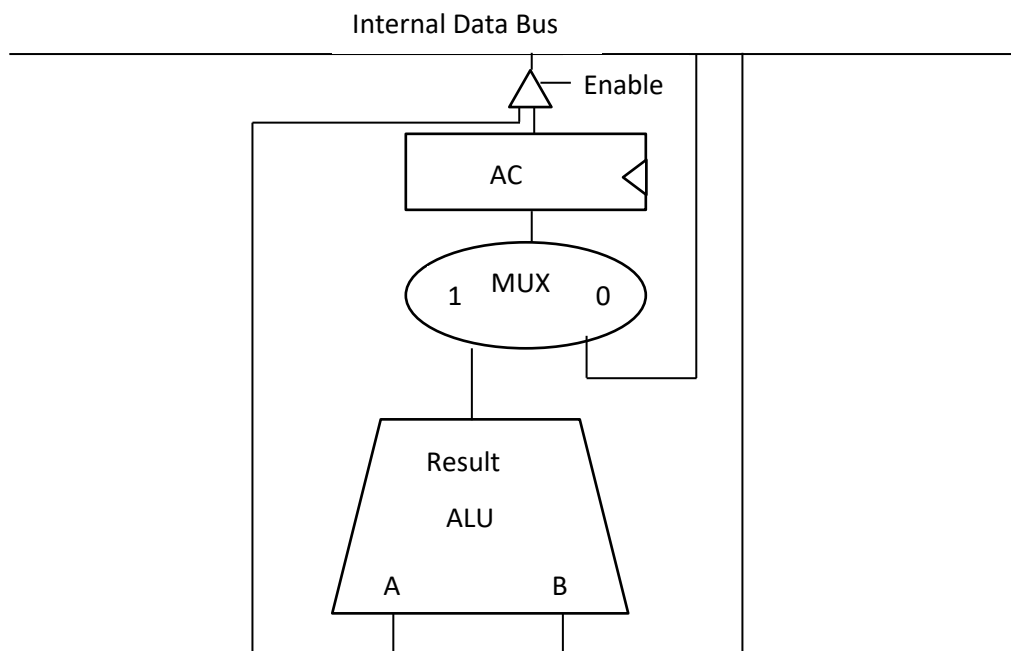
- a) Opcode: An opcode is a string that specifies the operation to be performed.
- b) Operand: An operand is the piece of an instruction that specifies the data to be operated on
- c) Effective Address: The effective address is a part of an instruction that acts like a pointer and refers to the actual operand in memory.
- d) Program Counter: The program counter is essentially a line counter, it keeps track of where in the code execution is currently at, it is used to fetch instructions, data, and more.

2 CPU with I address format and memory of 1k(1024). Supports 32 instructions. Instructions has three parts, opcode, indirect mode bit, and address field. If I-bit =0, the operand is located in M[Address], else the operand located in M[M[Address]]

Draw the instruction word format indicating the number of bits and the function of each field.

Opcode: 5 bits	I – Bit: 1 bit	Address: 10 bits
$2^5=32$, need to support 32 opcodes	Only needs to represent 1 or 0	$2^{10} = 1024$, need that many bits to address all of the memory space

3 Draw a block diagram of the hardware to connect C to the internal data bus and the ALU in the provided CPU, using a tri state buffer and a multiplexer.



4 Explain whether or not each of the following microoperations can be performed in a single clock cycle. PC and MAR each contain 12 bits and AC and MDR each contain 16bits and IR is 4 bits

- $MAR \leftarrow AC, MDR \leftarrow MAR$. No you can't simultaneously write the contents of MAR and read the contents of MAR in order to write them elsewhere.
- $IR \leftarrow MDR, MAR \leftarrow MDR$. Yes. Parallel write from MDR to IR and MAR.
- $MAR \leftarrow MDR, MDR \leftarrow M(MAR)$. No, reading and writing to MDR simultaneously.
- $MDR \leftarrow AC+1$. Yes, AC can increment itself and be read.
- $AC \leftarrow MDR, PC \leftarrow PC+1$. Yes two operations are separate and PC can increment itself
- $PC \leftarrow PC+AC$. No? Program counter cannot do arithmetic with the accumulator.

5 Give the microoperations required to implement the execute phase of STA (X)+. Given generic pseudo cpu from book. 16bit inst, 4bit opcode,12bit address. Operands are 16bits. PC and MAR are 12bits. AC,MDR,TEMP each contain 16bits. IR is 4 bits.

GIVEN FETCH

- $MAR \leftarrow PC$
- $MDR \leftarrow M(MAR), PC \leftarrow PC+1$
- $IR \leftarrow MDR(\text{Opcode}), MAR \leftarrow MDR(\text{address})$

Execute

- $MDR \leftarrow M(MAR), TEMP \leftarrow AC$
- $AC \leftarrow MDR$
- $AC \leftarrow AC+1$
- $MDR \leftarrow AC$
- $M(MAR) \leftarrow MDR, AC \leftarrow AC-1$
- $MAR \leftarrow AC$
- $MDR \leftarrow TEMP, AC \leftarrow TEMP$
- $M(MAR) \leftarrow MDR$

6

- Ldi r27,85

register	contents
x	0106->8506

Loading x high with 85

- Ror r2

register	contents
----------	----------

2	1b->8D
---	--------

Performing a rotate right through carry on r2, r2 is holding 1b-> 00011011, rotated:
10001101->8D

3. Adc r2,r1

register	contents
2	1b->20

Performs add with carry, result in r2. 1b+05(hex) =27+5(dec)=32(dec)=20(hex)

4. Sts \$0007,r28

Data Memory address	contents
\$0107	02->02

writes the value in YL into memory address \$0007. Seems like I made a mistake somewhere because the handout does not show a memory location \$0007. But I feel confident that STS \$XXXX Rs, is supposed to write the contents of register Rs to memory location \$XXXX.

5. Sbiw XH:XL, 2

Register	contents
x	0106->0104